

Sigma & YARA Rules

A. YARA

YARA adalah tool yang digunakan untuk mendeteksi dan mengklasifikasi malware berdasarkan pola tertentu (rules) yang ditulis dalam format teks. YARA banyak digunakan oleh malware analyst dan threat hunter.

Struktur Dasar YARA Rule

```
rule RuleName {  
    strings:  
        $a = "malicious_string"  
        $b = {6A 40 68 00 30 00 00 6A 14 8D 91}  
  
    condition:  
        $a or $b  
}
```

Penjelasan:

- **rule RuleName** : Nama rule (tidak boleh ada spasi).
- **strings** : Bagian yang mendefinisikan pola string atau hex yang dicari.
- **condition** : Syarat kapan rule akan dianggap match, misal jika salah satu string ditemukan.

Tipe Data dalam Strings:

- **Plain text**: Contoh: "MZ"
- **Hex pattern**: Contoh: {E8 ?? ?? ?? ?? 85 C0}
- **Regex**: Contoh: /malware[0-9]+/

YARA biasa digunakan untuk scan file secara lokal atau integrasi dengan sistem deteksi malware seperti VirusTotal, Wazuh, dan lainnya.

B. Yara Lab

1. setup

```
apt update  
apt install yara
```

2. cara menggunakan yara

- disini saya akan mencoba membuat file yang berisi string tertentu

```
mkdir test  
cd test  
  
echo "halo aku virus" > file.txt  
echo "1" > 1.txt && echo "2" > 2.txt && echo "3" > 3.txt && echo "4 virus"  
> 4.txt  
  
cd ..
```

- lalu saya akan coba buat sebuah rules yara untuk mendeteksi apakah terdapat string virus

```
cat << 'EOF' > test.yar  
rule string_virus  
{  
  strings:  
    $a = "virus"  
  condition:  
    $a  
}  
EOF
```

- jika sudah kita coba jalankan

```
yara test.yar test/
```

atau gunakan -s untuk melihat apa saja yang cocok

```
yara -s test.yar test/
```

```
root@ubuntuserver:/tmp# mkdir test
cd test

echo "halo aku virus" > file.txt
echo "1" > 1.txt && echo "2" > 2.txt && echo "3" > 3.txt && echo "4 virus" > 4.txt

cd ..
root@ubuntuserver:/tmp# cat << 'EOF' > test.yar
rule string_virus
{
    strings:
        $a = "virus"
    condition:
        $a
}
EOF
root@ubuntuserver:/tmp# yara test.yar test/
string_virus test//file.txt
string_virus test//4.txt
root@ubuntuserver:/tmp# yara -s test.yar test/
string_virus test//4.txt
0x2:$a: virus
string_virus test//file.txt
0x9:$a: virus
root@ubuntuserver:/tmp# |
```

C. Sigma

Sigma adalah format rule open-source untuk mendeteksi aktivitas mencurigakan dalam log (SIEM). Sigma bertujuan untuk menjadi "YARA untuk log".

flow sigma

```
Log mentah (raw logs dari Windows/Linux)
↓
Dikirim ke SIEM (Splunk, Elastic, dll)
↓
SIEM parsing dan indexing log
↓
Sigma Rule (format .yaml) dipakai → dikonversi ke query SIEM pakai sigma-cli
↓
Query dijalankan di SIEM → menghasilkan alert atau hasil pencarian
```

Struktur Dasar Sigma Rule

```
title: Suspicious PowerShell Execution
logsource:
  category: process_creation
  product: windows

detection:
  selection:
    Image|endswith: 'powershell.exe'
    CommandLine|contains: 'Invoke-WebRequest'
  condition: selection

level: high
```

Penjelasan:

- `title` : Nama rule.
- `logsource` : Menentukan jenis log yang dituju.
- `detection` : Pola yang ingin dicocokkan dalam log.
- `level` : Tingkat keparahan (low/medium/high/critical).

Sigma rules dapat dikonversi ke query SIEM seperti Splunk, Elastic, Graylog menggunakan Sigma CLI.

Keduanya, **YARA** dan **Sigma**, sangat penting dalam dunia **threat detection**:

- YARA: fokus ke *file-based detection*.
- Sigma: fokus ke *log-based detection*.

D. Sigma Lab

- sigma cli (msh gagal) github.com/SigmaHQ/sigma
- sigma_to_wazuh https://github.com/theflakes/sigma_to_wazuh?tab=readme-ov-file

1. setup

```
git clone https://github.com/theflakes/sigma_to_wazuh
cd sigma_to_wazuh
mkdir rules
```

lalu coba pip install, dan ubah config.ini untuk mengubah path rules sigma

```
pip3 install lxml bs4 ruamel.yaml --break-system-packages

nano config.ini
# untuk mengubah config path
# ubah ini jadi directory = ./rules agar bisa baca rulesnya

python3 sigma_to_wazuh.py
```

```
root@ubuntuserver:/tmp/sigma# git clone https://github.com/theflakes/sigma_to_wazuh
Cloning into 'sigma_to_wazuh'...
remote: Enumerating objects: 743, done.
remote: Counting objects: 100% (201/201), done.
remote: Compressing objects: 100% (91/91), done.
remote: Total 743 (delta 130), reused 178 (delta 110), pack-reused 542 (from 1)
Receiving objects: 100% (743/743), 237.22 KiB | 1.73 MiB/s, done.
Resolving deltas: 100% (490/490), done.
root@ubuntuserver:/tmp/sigma# cd sigma_to_wazuh
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# mkdir rules
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# pip3 install lxml bs4 ruamel.yaml --break-
Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (6.0.0)
Requirement already satisfied: bs4 in /usr/local/lib/python3.12/dist-packages (0.0.2)
Requirement already satisfied: ruamel.yaml in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: ruamel.yaml.clib>=0.2.7 in /usr/local/lib/python3.12/dis
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/di
WARNING: Running pip as the 'root' user can result in broken permissions and conflictin
warnings/venv
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# nano config.ini
```

```
GNU nano 7.2 config.ini *
[sigma]
# root of the Sigma rules URL
rules_link = https://github.com/SigmaHQ/sigma/tree/master/rules
# location of the Sigma rules directory
directory = ./rules
# file that Wazuh rules will be written to
out_file = ./sigma.xml
# convert sigma experimental rules (yes|no)
process_experimental = yes
# Sigma rule IDs to never try and convert
skip_sigma_guids = ()
```

2. testing untuk rules sigma (opsional ini contoh rules untuk deteksi sebuah process)

- ini rules wazuh untuk testing apakah lognya bisa ke detect (untuk lognya sendiri itu dari ai, semuanya dari AI :v)
- lakukan test ini di wazuh manager

```
cat << "EOF" > /var/ossec/etc/rules/local_rules.xml
<group name="windows,sysmon,">
<rule id="110000" level="10">
  <decoded_as>json</decoded_as>
  <field name="win.system.eventID">1</field>
  <field name="win.process.command_line">CreateProcessA</field>
  <field name="win.process.loaded_modules">WS2_32.dll</field>
  <description>Detect CreateProcessA + WS2_32.dll from
Sysmon</description>
</rule>
</group>
EOF

echo '{"win":{"system":{"eventID":1},"process":
{"command_line":"CreateProcessA","loaded_modules":"WS2_32.dll"}}}' |
/var/ossec/bin/wazuh-logtest
```

3. buat rules sigma yang mirip dengan rules wazuh di atas

- nah lalu kita buat sebuah rules dari yaml menggunakan sigma yang di convert ke wazuh nantinya sebagai xml

```
cat << "EOF" > ./rules/local.yml
title: Detect CreateProcessA and WS2_32.dll Usage
id: 6c0d0fd0-b91e-4c91-900a-f13faaa0ff01
description: Detects process creation with CreateProcessA and WS2_32.dll
loaded
status: experimental
author: Generated by ChatGPT
logsource:
  product: windows
  service: sysmon
  category: process_creation
detection:
  selection:
    CommandLine|contains: "CreateProcessA"
    ImageLoaded|endswith: "WS2_32.dll"
  condition: selection
fields:
  - CommandLine
  - Image
  - ParentImage
  - ImageLoaded
level: high
EOF

python3 sigma_to_wazuh.py
cat sigma.xml
```

```
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# cat << "EOF" > ./rules/local.yml
title: Detect CreateProcessA and WS2_32.dll Usage
id: 6c0d0fd0-b91e-4c91-900a-f13faaa0ff01
description: Detects process creation with CreateProcessA and WS2_32.dll loaded
status: experimental
author: Generated by ChatGPT
logsource:
  product: windows
  service: sysmon
  category: process_creation
detection:
  selection:
    CommandLine|contains: "CreateProcessA"
    ImageLoaded|endswith: "WS2_32.dll"
  condition: selection
fields:
  - CommandLine
  - Image
  - ParentImage
  - ImageLoaded
level: high
EOF
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# python3 sigma_to_wazuh.py
```

```
*****
Number of Sigma Experimental rules skipped: 0
Number of Sigma TIMEFRAME rules skipped: 0
Number of Sigma 1 OF with AND rules skipped: 0
Number of Sigma PAREN rules skipped: 0
Number of Sigma CIDR rules skipped: 0
Number of Sigma NEAR rules skipped: 0
Number of Sigma CONFIG rules skipped: 0
Number of Sigma ERROR rules skipped: 0
-----
Total Sigma rules skipped: 0
Total Sigma rules converted: 1
-----
Total Wazuh rules created: 1
-----
Total Sigma rules: 1
Sigma rules converted %: 100.0
*****
```

```
root@ubuntuserver:/tmp/sigma/sigma_to_wazuh# cat sigma.xml
<group name="sigma">
  <!--
Author: Brian Kellogg
Sigma: https://github.com/SigmaHQ/sigma
Wazuh: https://wazuh.com
All Sigma rules licensed under DRL: https://github.com/SigmaHQ/sigma/blob/master/LICENSE.Detection.Rules.md
-->
  <rule id="900001" level="13">
    <info type="link">https://github.com/SigmaHQ/sigma/tree/master/rules/rules/local.yml</info>
    <!--Sigma Rule Author: Generated by ChatGPT-->
    <!--Description: Detects process creation with CreateProcessA and WS2_32.dll loaded-->
    <!--Status: experimental-->
    <!--ID: 6c0d0fd0-b91e-4c91-900a-f13faaa0ff01-->
    <description>Detect CreateProcessA and WS2_32.dll Usage</description>
    <options>no_full_log</options>
    <group>windows,sysmon,process_creation,</group>
    <if_sid>184665, 185000, 185001, 185002, 185003, 185004, 185005, 185006, 185007, 185008, 185009, 185010,
4716, 184717, 184726, 184727, 184736, 184737, 184746, 184747, 184766, 184767, 184776</if_sid>
    <field name="win.eventdata.commandLine" negate="no" type="pcre2">(i)CreateProcessA</field>
    <field name="win.eventdata.imageLoaded" negate="no" type="pcre2">(i)WS2_32\.dll</field>
  </rule>
```

4. testing rules xml menggunakan wazuh

- lakukan **copy paste output sigma.xml** yang dihasilkan **sigma_to_wazuh**, dan lakukan **test dengan log di atas tadi**

```
cat << "EOF" > /var/ossec/etc/rules/local_rules.xml
```

```
## paste output hasil dari cat sigma.xml
```

```
EOF
```

```
echo '{"win":{"system":{"eventID":1},"process":
```

```
{"command_line":"CreateProcessA","loaded_modules":"WS2_32.dll"}}}' |
```

```
/var/ossec/bin/wazuh-logtest
```

```
[root@wazuh-server rules]# cat << "EOF" > /var/ossec/etc/rules/local_rules.xml
> <group name="sigma,">
  <!--
Author: Brian Kellogg
Sigma: https://github.com/SigmaHQ/sigma
Wazuh: https://wazuh.com
All Sigma rules licensed under DRL: https://github.com/SigmaHQ/sigma/blob/master/LICENSE.Detection.Rules.md
-->
  <rule id="900001" level="13">
    <info type="link">https://github.com/SigmaHQ/sigma/tree/master/rules/rules/local.yml</info>
    <!--Sigma Rule Author: Generated by ChatGPT-->
    <!--Description: Detects process creation with CreateProcessA and WS2_32.dll loaded-->
    <!--Status: experimental-->
    <!--ID: 6c8d0fd0-b91e-4c91-900a-f13faaa0fff01-->
    <description>Detect CreateProcessA and WS2_32.dll Usage</description>
    <options>no_full_log</options>
    <group>windows,sysmon,process_creation,</group>
    <if_sid>184665, 185000, 185001, 185002, 185003, 185004, 185005, 185006, 185007, 185008, 185009, 185010, 185011, 185012, 185013, 184666, 184667, 184676, 184677, 184678, 184698, 184706, 184707, 184716, 184717, 184726, 184727, 184736, 184737, 184746, 184747, 184766, 184767, 184776</if_sid>
    <field name="win.eventdata.commandLine" negate="no" type="pcrc2">(?!CreateProcessA</field>
    <field name="win.eventdata.imageLoaded" negate="no" type="pcrc2">(?!WS2_32.dll</field>
  </rule>
</group>
> EOF
[root@wazuh-server rules]# echo '{"win":{"system":{"eventID":1},"process":{"command_line":"CreateProcessA","loaded_modules":"WS2_32.dll"}}}' | /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.12.0
Type one log per line

**Phase 1: Completed pre-decoding.
full event: '{"win":{"system":{"eventID":1},"process":{"command_line":"CreateProcessA","loaded_modules":"WS2_32.dll"}}}'

**Phase 2: Completed decoding.
name: 'json'
win.process.command_line: 'CreateProcessA'
win.process.loaded_modules: 'WS2_32.dll'
win.system.eventID: '1'

[root@wazuh-server rules]#
```

Ruler dari Malware

A. Yara

1. cara buat rules yara menggunakan chat gpt

- Dapatkan sample malware (contoh: .exe, .dll, .bin)
- Jalankan `strings namafile` untuk melihat isi string mencurigakan
- (Opsional) Upload ke VirusTotal untuk cek SHA256 dan behavior
- Salin hasil `strings` ke ChatGPT untuk dibuatkan YARA rule
- Simpan dan tes dengan `yara rule.yar namafile`

2. setup

```
git clone https://github.com/karetnyadua123/tugas-week-8
```

3. buat rules yang sudah dibuat berdasarkan IOC (Indicators of Compromise)

- [./local.yar](#)

```
1 rule Malware_Lab01_01_Generic
2 {
3     meta:
4         description = "Generic detection for Lab01-01.dll malware"
5         author = "Generated by ChatGPT"
6         type = "Win32 DLL"
7         sha256 = "f50e42c8dffaab649bde0398867e930b86c2a599e8db83b8260393082268f2dba"
8
9     strings:
10         $mz = { 4D 5A } // MZ header
11         $s1 = "CreateProcessA"
12         $s2 = "CreateMutexA"
13         $s3 = "OpenMutexA"
14         $s4 = "CloseHandle"
15         $s5 = "WS2_32.dll"
16         $s6 = "exec"
17         $s7 = "sleep"
18         $s8 = "hello"
19         $s9 = "127.26.152.13"
20         $s10 = "SADFHJHF"
21
22     condition:
23         $mz at 0 and
24         6 of ($s*)
25 }
26
27 rule Lab01_01_exe
28 {
29     meta:
30         description = "Detection for Lab01-02.bin malware"
31         author = "Generated by ChatGPT"
32         type = "Win32 EXE"
33         sha256 = "f50e42c8dffaab649bde0398867e930b86c2a599e8db83b8260393082268f2dba"
34
35     strings:
36         $mz = "This program cannot be run in DOS mode." ascii
37         $s1 = "WARNING_THIS_WILL_DESTROY_YOUR_MACHINE" ascii
38         $s2 = "CopyFileA" ascii
39         $s3 = "kernel32.dll" ascii
40         $s4 = "Lab01-01.dll" ascii
41         $dll_ref = "C:\\windows\\system32\\kernel32.dll" ascii
42         $api1 = "CreateFileMappingA" ascii
43         $api2 = "MapViewOfFile" ascii
44         $strcasecmp = "_stricmp" ascii
```

```
cat << "EOF" > local.yar
## copy paste rule di link di atas
EOF
```

4. testing rulesnya ke directory tugas_week_8

```
yara local.yar tugas-week-8/

yara -s local.yar tugas-week-8/Lab01-01.dll
yara -s local.yar tugas-week-8/Lab01-01.exe
yara -s local.yar tugas-week-8/Lab01-02.bin
```

atau gunakan -s

```
yara -s local.yar tugas-week-8/
```

```
root@ubuntuserver:/tmp# git clone https://github.com/karetnyadua123/tugas-week-8
Cloning into 'tugas-week-8'...
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (5/5), 6.61 KiB | 6.61 MiB/s, done.
Resolving deltas: 100% (1/1), done.
root@ubuntuserver:/tmp# nano local.yar
root@ubuntuserver:/tmp# ls tugas-week-8/
Lab01-01.dll Lab01-01.exe Lab01-02.bin
root@ubuntuserver:/tmp# yara local.yar tugas-week-8/
Lab01_01_exe tugas-week-8//Lab01-01.exe
Malware_Lab01_01_Generic tugas-week-8//Lab01-01.dll
Malware_Lab01_02_Generic tugas-week-8//Lab01-02.bin
root@ubuntuserver:/tmp# yara -s local.yar tugas-week-8/Lab01-01.dll
Malware_Lab01_01_Generic tugas-week-8/Lab01-01.dll
0x0:$mz: 4D 5A
0x2120:$s1: CreateProcessA
0x2132:$s2: CreateMutexA
0x2142:$s3: OpenMutexA
0x210a:$s4: CloseHandle
0x215c:$s5: WS2_32.dll
0x26010:$s6: exec
0x26018:$s7: sleep
0x26020:$s8: hello
0x26028:$s9: 127.26.152.13
0x26038:$s10: SADFHUF
root@ubuntuserver:/tmp# yara -s local.yar tugas-week-8/Lab01-01.exe
Lab01_01_exe tugas-week-8/Lab01-01.exe
0x4e:$mz: This program cannot be run in DOS mode.
0x30b0:$s1: WARNING_THIS_WILL_DESTROY_YOUR_MACHINE
0x21b8:$s2: CopyFileA
0x3010:$s3: kerne132.dll
0x3060:$s3: kerne132.dll
0x307c:$s4: Lab01-01.dll
0x304c:$dll_ref: C:\windows\system32\kerne132.dll
0x2166:$api1: CreateFileMappingA
0x2156:$api2: MapViewOfFile
0x22a8:$strcasecmp: _stricmp
0x21c2:$lib1: KERNEL32.dll
0x21e2:$lib2: MSVCRT.dll
root@ubuntuserver:/tmp# yara -s local.yar tugas-week-8/Lab01-02.bin
Malware_Lab01_02_Generic tugas-week-8/Lab01-02.bin
0x0:$mz: 4D 5A
0x1d8:$upx0: UPX0
0x200:$upx1: UPX1
0x228:$upx2: UPX2
0x5fb:$malstring1: MalService
0x628:$book: ysisbook.co
0x639:$exploit: Int6net Explo!r 8FEI
0x611:$url: http://w
0xa98:$kernel: KERNEL32.DLL
0xaa5:$advapi: ADVAPI32.dll
0xabd:$wininet: WININET.dll
0xaca:$loadlib: LoadLibraryA
0xad8:$getproc: GetProcAddress
0xaf8:$virtualloc: VirtualAlloc
0xae8:$virtprot: VirtualProtect
0xb14:$exitproc: ExitProcess
```

